

DATA STRUCTURES AND ALGORITHMS PROJECT

LegalSearch

Intelligent Legal Document Analytics Platform

Student name: [Editable placeholder]
Roll number: [Editable placeholder]
Department / College: [Editable placeholder]
Guide: [Editable placeholder]

Made with GAMMA

THE PROBLEM

LegalSearch turns manual legal search into an algorithmic workflow

Large collections of legal documents make manual searching slow and difficult to scale.

Explore

Search, indexing, text comparison, and assignment algorithms in a legal-document setting.

Demonstrate

An educational prototype, not production legal advice.

Organize

Register documents, retrieve relevant text, compare wording, and show case-assignment logic.

Made with GAMMA

PROJECT MAP

Six DSA modules make the prototype useful and teachable



Search

Keyword and clause search with multiple string algorithms.



Indexing

Text indexing and substring retrieval with suffix structures.



Comparison

Legal wording comparison and optimization examples with dynamic programming.



Assignment

Lawyer-to-case assignment using network flow and matching.



Hard problems

Computationally difficult selection problems with NP-completeness and approximation.



Scale

Randomized processing and parallel computation demonstrations.

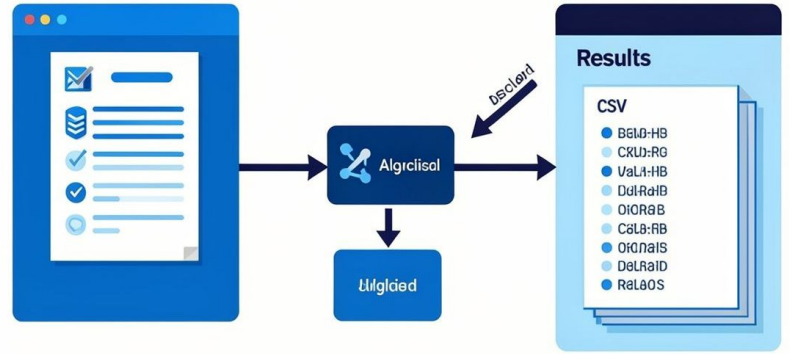
Case and document information is stored and managed as structured records.

Made with GAMMA

SYSTEM ARCHITECTURE

One Java application connects users, algorithms, and records

1. Register documents and case information.
2. Search keywords or clauses through a console or browser interface.
3. Route requests through shared Java application logic.
4. Execute the selected DSA module against CSV-backed data.
5. Display search results, comparisons, assignments, and reports.



Made with GAMMA

String algorithms make clause and keyword search precise

KMP: Avoids redundant comparisons during pattern matching.

Z-Function: Exposes matching prefixes for fast pattern analysis.

Rabin–Karp: Uses rolling hashes to compare candidate matches.

Aho–Corasick: Searches multiple terms such as clauses, case references, and keywords.

Illustrative Java classes: **KMP**, **ZFunction**, **RabinKarp**, **AhoCorasick**.

EDITABLE MATCHING STRIP

Active algorithm: Aho–Corasick

“The court may terminate the agreement if the party fails to perform.”

court | terminate | agreement

Highlighted matches can represent clauses, case references, or keywords.

Made with GAMMA

MODULE 2 · INDEXING

Suffix structures turn legal text into searchable indexes

Index long documents, retrieve substrings, and surface recurring legal phrases.

Indexing structures

- Suffix Array and SA-IS build ordered suffix-based indexes.
- LCP Array with Kasai's algorithm measures shared prefixes between adjacent suffixes.
- Suffix Trees and Suffix Automata support substring retrieval and repeated-phrase analysis.

Illustrative Java classes: SuffixArray, **SAIS**, **KasaiLCP**, **SuffixTree**, **SuffixAutomaton**.

EDITABLE SUFFIX-ARRAY VIEW · DOCUMENT TEXT: "CONTRACT CLAUSE"

Rank	Sorted suffix	Document position
1	clause	9
2	contract clause	1
3	t clause	8

Made with GAMMA

Dynamic programming compares wording without overstating legal meaning

Levenshtein counts insertions, deletions, and substitutions; **Damerau–Levenshtein** also counts transpositions where applicable.

Weighted edit distance assigns different costs to different changes.

Applications: typo correction, wording comparison, and algorithmic optimization examples. Bitmask dynamic programming and matrix-chain multiplication demonstrate optimization patterns.

Boundary: similarity does not establish legal equivalence.

EDIT-DISTANCE GRID

“contract” → “contact”

	∅	c	o	n	t	a	c	t
∅	0	1	2	3	4	5	6	7
c	1	0	1	2	3	4	5	6
o	2	1	0	1	2	3	4	5
n	3	2	1	0	1	2	3	4
t	4	3	2	1	0	1	2	3
r	5	4	3	2	1	1	2	3
a	6	5	4	3	2	1	2	3
c	7	6	5	4	3	2	1	2
t	8	7	6	5	4	3	2	1

Levenshtein distance: 1 (delete “r”).

Made with GAMMA



MODULE 4 · NETWORK FLOW

Network flow models lawyer-to-case assignment

Eligibility and capacity constraints become a graph problem: which assignments can be made, and how many?

Lawyer A

Eligible: Case 1, Case 2

Lawyer B

Eligible: Case 2, Case 3

Lawyer C

Eligible: Case 1, Case 3

Highlighted assignments

Lawyer A → Case 1 · Lawyer B → Case 2 · Lawyer C → Case 3

Ford–Fulkerson, Edmonds–Karp, and Dinic's algorithm demonstrate maximum-flow strategies. König's theorem connects matching and vertex cover.

Made with GAMMA

MODULES 5–6

Advanced modules broaden the project beyond search

Two extensions connect LegalSearch to difficult selection problems, randomized processing, and scalable computation.

MODULE 5

NP-Completeness and Approximation

Concepts: SAT and 3-SAT; 3-SAT → CLIQUE; CLIQUE → INDEPENDENT SET → VERTEX COVER.

Practice: Vertex-Cover 2-approximation demonstrates practical handling of difficult selection problems.

Application: conceptual legal-resource selection problems.

MODULE 6

Randomized and Parallel Algorithms

Randomized: Randomized QuickSort, Reservoir Sampling, and Miller–Rabin primality testing.

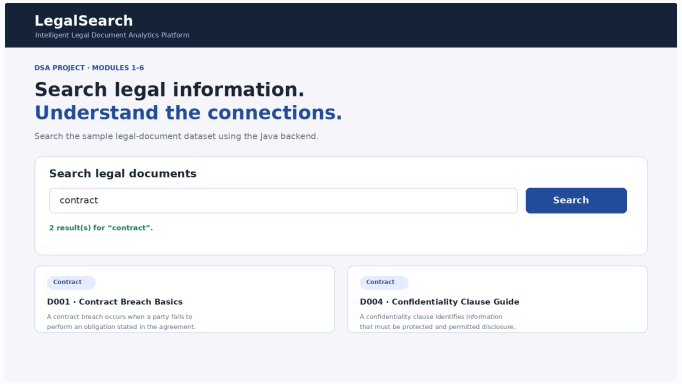
Parallel: Blelloch Scan, Parallel Reduce, and Brent's theorem introduce parallel computation.

Application: randomized data processing and scalable computation concepts.

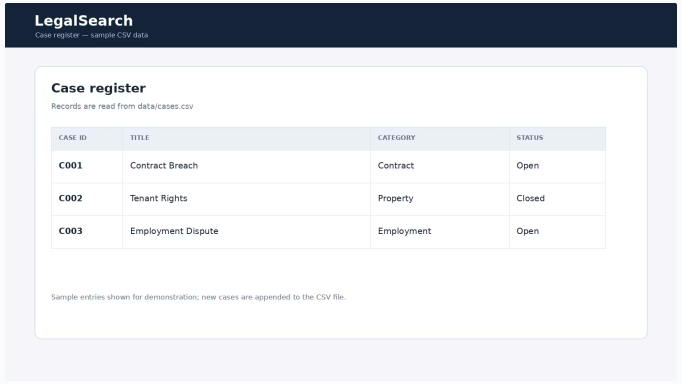
Made with GAMMA

CONCLUSION

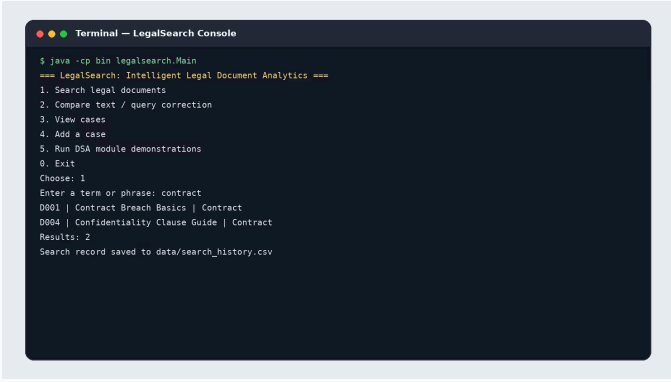
Implemented outputs



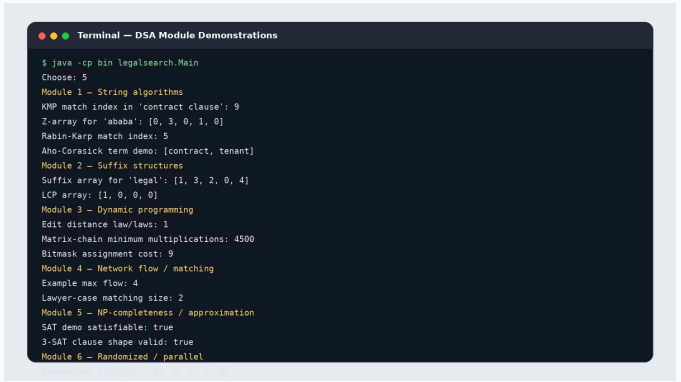
Website search results



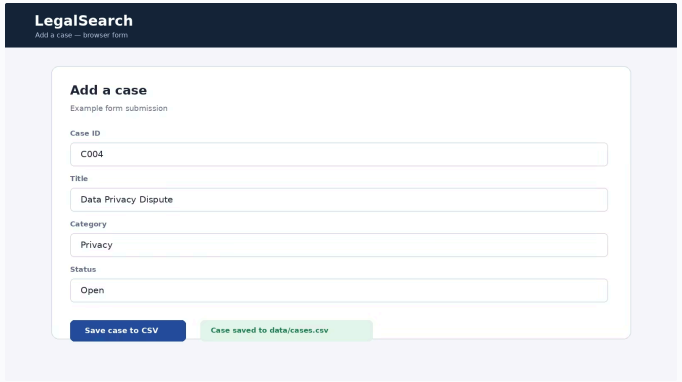
Case register



Console search output



DSA module demonstrations



Add-case output

EDUCATIONAL VALUE

Connects abstract DSA techniques to document management. Algorithms organize and analyze legal documents, not legal judgment.

Future scope: PDF/DOCX text extraction, database integration, authentication, larger datasets, and deployment.

Made with GAMMA